

Format penjelasan Subbab 3.2.4.1 berdasarkan transformasi struktur 4.1.3 ke tiga grup diagram.

Pembangunan Komponen *Hybrid Retrieval*

Komponen *hybrid retrieval* dibangun untuk menggabungkan pencarian semantik berbasis *embedding* dan pencarian leksikal berbasis BM25. Dalam rancangan Bab 3, proses ini dapat dijelaskan melalui tiga grup utama yang sesuai dengan diagram, yaitu Basis Pengetahuan, Pemrosesan *Query*, dan Pemrosesan *Scoring*. Ketiga grup tersebut menunjukkan alur dari pembentukan representasi dokumen, pemrosesan masukan pengguna, hingga penggabungan skor untuk memilih dokumen paling relevan.

Basis Pengetahuan

Grup Basis Pengetahuan merupakan fase inisialisasi yang dijalankan saat sistem dimulai untuk menyiapkan seluruh representasi dokumen kontrol ISO. Pada tahap ini, data kontrol ISO dimuat dari berkas JSON, dikonversi menjadi representasi teks, kemudian diproses melalui jalur *dense* untuk membentuk *embedding* dokumen dan jalur *sparse* untuk membangun indeks BM25. Hasil dari fase ini disimpan dalam memori agar dapat digunakan kembali ketika *query* pengguna diproses.

- a. **`_load_controls (controls_path)`**: Proses ini memuat data basis pengetahuan dari berkas JSON yang berisi 93 kontrol ISO. Hasil pemuatan data menjadi daftar kontrol yang akan digunakan sebagai korpus dokumen pada proses *retrieval*.
- b. **`control_to_text (control=control)`**: Proses ini mengubah satu data kontrol ISO berbentuk JSON menjadi representasi teks datar (*flat text*). Representasi teks ini menggabungkan atribut penting seperti judul, objektif, deskripsi, serta atribut pengayaan yang tersedia agar dokumen memiliki konteks yang lebih lengkap.
- c. **`encode_texts (texts=control_texts)`**: Proses ini mengubah seluruh representasi teks kontrol ISO menjadi vektor *embedding*. Vektor ini merepresentasikan makna semantik dokumen sehingga dapat dibandingkan dengan *query* pada ruang vektor yang sama.
- d. **`normalize (vectors=doc_embeddings)`**: Proses ini melakukan normalisasi L2 pada vektor *embedding* dokumen. Normalisasi dilakukan agar perhitungan *dot product* antara *query* dan dokumen setara dengan *cosine similarity*.

- e. **HybridRetriever.doc_embeddings**: Proses ini menyimpan vektor *embedding* dokumen pada atribut `doc_embeddings`. Penyimpanan dilakukan secara *in-memory* sehingga indeks vektor dapat digunakan langsung saat sistem menghitung *dense score*.
- f. **tokenize(text=control_text)**: Proses ini mengubah representasi teks dokumen menjadi daftar token leksikal. Tokenisasi dilakukan dengan mengubah teks menjadi huruf kecil dan mengambil karakter alfanumerik agar dokumen siap diproses oleh BM25.
- g. **BM25Retriever(documents=control_texts)**: Proses ini menginisialisasi indeks BM25 berdasarkan seluruh teks kontrol ISO. Inisialisasi ini membentuk struktur data yang diperlukan untuk menghitung kecocokan kata kunci antara *query* dan dokumen.
- h. **_compute_doc_freq(tokenized_docs)**: Proses ini menghitung jumlah dokumen yang mengandung setiap *term*. Nilai *document frequency* digunakan sebagai dasar untuk menentukan seberapa umum atau spesifik suatu *term* dalam korpus.
- i. **_compute_idf(doc_freq)**: Proses ini menghitung nilai *inverse document frequency* untuk setiap *term*. Nilai IDF memberi bobot lebih tinggi pada *term* yang lebih jarang muncul sehingga dokumen dengan kata kunci spesifik dapat diberi skor lebih relevan.
- j. **BM25 index**: Proses ini menyimpan informasi *term frequency*, panjang dokumen, rata-rata panjang dokumen, dan IDF. Informasi tersebut digunakan kembali pada tahap Pemrosesan *Query* untuk menghitung *sparse score*.

Pemrosesan *Query*

Grup Pemrosesan *Query* dijalankan setiap kali pengguna memasukkan pertanyaan atau kebutuhan informasi. Pada tahap ini, *query* diproses melalui dua jalur yang sama dengan dokumen, yaitu jalur *dense* untuk menghasilkan skor kemiripan semantik dan jalur *sparse* untuk menghasilkan skor kecocokan kata kunci. Hasil dari tahap ini berupa *dense score* dan *sparse score* untuk seluruh kontrol ISO.

- a. **encode_texts(texts=[query])**: Proses ini mengubah *query* pengguna menjadi vektor *embedding*. Karena fungsi yang sama digunakan pada dokumen dan *query*, keduanya berada pada ruang representasi semantik yang sama.
- b. **normalize(vectors=query_embedding)**: Proses ini menormalkan vektor *query* menggunakan normalisasi L2. Normalisasi membuat perhitungan kemiripan antara *query* dan dokumen dapat dilakukan secara konsisten menggunakan *dot product*.

- c. **np.dot(doc_embeddings, q_emb)**: Proses ini menghitung hasil *dot product* antara vektor *query* dan seluruh *embedding* dokumen. Hasilnya adalah *dense score* yang merepresentasikan tingkat kemiripan semantik *query* terhadap 93 kontrol ISO.
- d. **tokenize(text=query)**: Proses ini mengubah *query* menjadi daftar token leksikal. Tokenisasi *query* menggunakan aturan yang sama dengan dokumen agar pencocokan kata kunci pada BM25 berjalan konsisten.
- e. **score_query(query=query)**: Proses ini menghitung skor BM25 antara *query* dan seluruh dokumen. Skor dihitung berdasarkan kombinasi *term frequency*, IDF, dan normalisasi panjang dokumen sehingga menghasilkan *sparse score* untuk 93 kontrol ISO.

Pemrosesan Scoring

Grup Pemrosesan *Scoring* berfungsi untuk menggabungkan *dense score* dan *sparse score* menjadi satu skor akhir. Kedua jenis skor terlebih dahulu dinormalisasi karena memiliki skala yang berbeda. Setelah itu, skor digabungkan menggunakan *weighted sum*, diurutkan, dan digunakan untuk memilih dokumen teratas sebagai hasil *retrieval*.

- a. **_minmax(scores=dense_scores)**: Proses ini menormalisasi *dense score* ke rentang 0 sampai 1. Normalisasi ini diperlukan agar skor semantik memiliki skala yang sebanding dengan skor BM25.
- b. **_minmax(scores=sparse_scores)**: Proses ini menormalisasi *sparse score* ke rentang 0 sampai 1. Normalisasi dilakukan secara terpisah dari *dense score* karena kedua skor berasal dari metode yang berbeda.
- c. **weighted_sum(dense_norm, sparse_norm)**: Proses ini menggabungkan skor *dense* dan *sparse* yang telah dinormalisasi. Formula yang digunakan adalah $hybrid_score = w_{dense} \times dense_norm + w_{sparse} \times sparse_norm$.
- d. **np.argsort(a=-hybrid_scores)[:3]**: Proses ini mengurutkan dokumen berdasarkan *hybrid score* secara menurun. Tiga dokumen dengan skor tertinggi dipilih sebagai kandidat paling relevan.
- e. **_build_result(idx=top_indices)**: Proses ini mengambil metadata dokumen terpilih seperti *control_id*, *title*, *objective*, dan *description*. Metadata beserta skor *dense*, *sparse*, dan *hybrid* digunakan sebagai hasil akhir tahap *retrieval*.